

[AI 기반 주차난 해결 서비스]

프로젝트 관리 계획서

GitHub Public Repository 링크:

<https://github.com/csy5271abcd/software-engineering-ai-parking-service>

문서번호 : [최수연]프로젝트관리계획서_260413_Doc-001

소 속 : 소프트웨어학과

팀 명 : SmartPark

팀 원 : 2023121183 최수연

제/개정 이력

[illegible]

목 차

1. 서론	1
1.1 문서의 목적 및 범위	1
1.2 프로젝트 개요	1
1.3 프로젝트 목표	2
1.4 용어 정의	2
1.5 참조 문서	2
2. 개발 계획	3
2.1 개발 절차 모형	3
2.2 WBS(Work Breakdown Structure)	3
2.3 개발 내용	4
2.4 개발 일정	5
3. 팀 구성	6
3.1 팀 구조	6
3.2 역할 및 책임	6
4. 품질 관리	7
4.1 팀 미팅 계획	7
4.2 변경 사항 관리 방법	7
4.3 산출물 관리 방법	8
5. 개발 환경	9
5.1 하드웨어 환경	9
5.2 소프트웨어 환경	9
5.3 기타 개발 환경	9
6. 산출물	10
6.1 산출물 정의	10
6.2 산출물 작성일 및 담당자	10
6.3 기타 제약사항	11
7. 기타 사항	11
7.1 리스크 관리	11
8. 참고문헌 및 부록	11

1. 서 론

1.1 문서 목적 및 범위

본 프로젝트 관리 계획서는 SmartPark의 AI 기반 주차난 해결 서비스에 대한 전반적인 내용을 체계적으로 정리하여, 프로젝트를 효율적으로 수행하기 위한 기준 문서로 활용하는 것을 목적으로 한다. 본 문서는 프로젝트 수행 과정에서 필요한 주요 관리 요소를 정의하고, 개발 진행 방향과 관리 방법을 일관성 있게 유지하는 데 목적이 있다.

본 문서는 다음과 같은 범위의 지표로 적용할 수 있다.

- ① 프로젝트 개요 (프로젝트 정의, 주요 기능 정리)
- ② 개발 계획 및 환경 (개발 절차 모형, 내용 및 일정, 소프트웨어/하드웨어 환경)
- ③ 팀 구성 (팀 구조/역할 및 책임)
- ④ 품질관리 (팀 미팅, 변경 사항 및 산출물 관리)
- ⑤ 산출물 (산출물 정의, 작성일 및 담당자)

즉, 본 계획서는 SmartPark 프로젝트의 전체적인 추진 방향을 정리하고, 개발 과정에서 발생할 수 있는 일정 관리, 역할 분담, 품질 관리, 형상 관리 등을 종합적으로 관리하기 위한 기준 문서이다.

1.2 프로젝트 개요

1.2.1 프로젝트 정의

AI 기반 주차난 해결 서비스 SmartPark는 도심 및 주거 지역에서 발생하는 주차 문제를 해결하기 위한 스마트 주차 서비스이다. 본 서비스는 사용자의 현재 위치와 목적지를 기반으로 주변 주차 가능 공간을 탐색하고, 곧 비워질 주차 공간 정보와 개인 소유 주차장의 공유 기능을 함께 제공하여 주차 공간 탐색의 효율성을 높이는 것을 목표로 한다.

또한 본 프로젝트는 React Native 기반 모바일 애플리케이션, Spring Boot 기반 백엔드, MySQL 데이터베이스, 그리고 Naver Maps API(Dynamic Map, GeoCoding, Reverse GeoCoding, Directions5 등)를 활용하여 구현한다. 지도 기반 탐색, 주소 변환, 경로 안내와 같은 핵심 기능을 시스템에 통합하고, AI 기반 혼잡도 분석을 통해 사용자가 실제로 주차 성공 가능성이 높은 공간을 선택할 수 있도록 지원한다.

아울러 본 프로젝트의 구현 과정에서는 Claude Code와 Codex를 활용한 바이트 코딩 방식을 반영하여, 기능 설계와 구현, 수정 과정을 보다 빠르고 유연하게 진행하고자 한다.

1.2.2 주요 기능 설명

ID	주요 기능	설명
F01	현재 위치 기반 주차 공간 조회	사용자의 현재 위치를 기준으로 주변 주차 가능 공간을 지도에 표시하고, 거리·요금·운영시간·이용 가능 여부를 제공한다.
F02	곧 비워질 주차 공간 확인	이용자의 출차 예정 시간을 바탕으로 곧 이용 가능한 주차 공간 정보를 제공한다.
F03	주차장 공급자 등록	개인 소유자가 자신의 주차 공간 정보를 등록하고, 유휴 시간대에 공유할 수 있도록 지원한다.
F04	NFC 기반 간편 결제	NFC를 통해 주차 이용 시작·종료 및 요금 결제를 간편하게 처리한다.
F05	AI 기반 혼잡도 분석	도착 예정 시간, 목적지, 과거 데이터 등을 바탕으로 주차장 혼잡도를 예측하고 추천한다.

1.3 프로젝트 목표

본 프로젝트의 목표는 도심 및 주거 지역에서 발생하는 주차난 문제를 해결할 수 있는 AI 기반 주차 서비스 SmartPark를 개발하는 것이다. 사용자의 현재 위치와 목적지를 기반으로 주변 주차 가능 공간을 효율적으로 탐색할 수 있도록 지원하고, 곧 비워질 주차 공간 정보, 개인 주차장 공유 기능, NFC 기반 간편 결제 기능, AI 기반 혼잡도 분석 기능을 제공하여 주차 탐색 시간과 불편을 줄이는 것을 목표로 한다. 또한 본 프로젝트는 React Native, Spring Boot, MySQL, Naver Maps API를 활용하여 실제 서비스 형태의 시스템을 구현하고, 사용자와 주차 공간 공급자를 연결하는 스마트 주차 플랫폼을 구축하는 것을 목적으로 한다.

1.4 용어 정의

본 문서의 이해를 돕기 위해 사용된 모든 용어 및 약어를 설명하고 정의합니다.

용어	설명
API	응용 프로그램 간 기능이나 데이터를 주고받을 수 있도록 제공되는 인터페이스
React Native	하나의 코드베이스로 모바일 애플리케이션을 개발할 수 있는 프레임워크
Naver Maps API	지도 표시, 주소 변환, 경로 탐색 등 위치 기반 기능을 제공하는 네이버 지도 API
Dynamic Map	지도 화면에 위치, 마커, 경로 등을 시각적으로 표시하는 지도 기능
GeoCoding	주소를 위도·경도 좌표로 변환하는 기능
Reverse GeoCoding	위도·경도 좌표를 실제 주소 정보로 변환하는 기능
Directions5	출발지와 목적지 사이의 경로 및 이동 정보를 제공하는 길찾기 기능
NFC	가까운 거리에서 기기 간 데이터를 주고받을 수 있는 근거리 무선 통신 기술
AI 혼잡도 분석	시간대, 위치, 과거 이용 데이터 등을 바탕으로 주차장 혼잡도를 예측하는 기능
바이브 코딩	Claude Code, Codex 등 AI 도구를 활용하여 설계·구현·수정을 빠르게 진행하는 개발 방식

1.4 참조 문서

[최수연]시스템정의서_260313_Doc-001.pdf

https://drive.google.com/file/d/1PGp9msmd13l4JSmHetwzTGD25mSPFSRp/view?usp=drive_link

2. 개발 계획

2.1 개발 절차 모형

1) 소프트웨어를 개발하기 위한 개발 절차 모형

본 프로젝트는 지도 API 연동, 주차 공간 데이터 처리, AI 혼잡도 분석 등 다양한 기능이 포함되어 있어 개발 과정에서 요구사항의 수정과 보완이 발생할 가능성이 높다. 따라서 변화에 유연하게 대응하고, 기능을 짧은 주기로 개발·점검할 수 있도록 Agile 방법론의 Scrum 모델을 적용한다. 또한 구현 과정에서는 Claude Code와 Codex를 활용한 바이트 코딩 방식을 반영하여 개발 효율을 높이하고자 한다.

2) 분석, 설계를 진행하기 위한 방법론

본 프로젝트는 모바일 앱, 백엔드 서버, 데이터베이스, 외부 지도 API가 연계된 구조이므로, 각 기능을 명확하게 구분하고 확장 가능하도록 설계할 필요가 있다. 이를 위해 객체지향 방법론을 기반으로 분석과 설계를 진행하며, React Native 프론트엔드, Spring Boot 백엔드, MySQL 데이터베이스, Naver Maps API 연동 구조를 중심으로 시스템을 구성한다.

2.2 WBS(Work Breakdown Structure)

번호	단계	세부 작업
1	요구분석	프로젝트 목표 정의, 기존 서비스 조사, 요구사항 분석, 주요 기능 정의
2	설계	시스템 구조 설계, 앱 구조 설계, DB 설계, API 연동 설계, 상세 기능 설계
3	구현	React Native 앱 구현, Spring Boot 서버 구현, MySQL 구축, Naver Maps API 연동, 핵심 기능 구현
4	테스트	단위 테스트, 기능 테스트, 통합 테스트, 오류 수정
5	문서화	산출물 작성, Git 형상관리, 발표 자료 작성

2.3 개발 활동

개발 활동	설명
프로젝트 요약	프로젝트 요약서 작성
프로젝트 계획	프로젝트 계획서 작성
요구사항 정의	요구사항 정의서 작성
요구사항 분석	요구사항 분석서 작성
Product Backlog	요구사항을 우선순위대로 순서화
Sprint Planning Meeting	스프린트가 시작될 때마다 백로그 선정 및 계획
Daily Scrum Meeting	매일 오전 진행상황과 이슈 사항, 계획 공유
Sprint	1~4주 단위로 짧게 프로그램을 개발하면서 개발자들끼리 피드백 제공
Sprint Retrospective	스프린트 기간 동안 팀 내부에서 발생한 일들을 회고하며 개선할 것, 잘한 것, 새로 시작할 것을 도출
Sprint Review	스프린트 종료 시점에 팀원 전체가 각자 한 일 리뷰 및 문서화

2.4 개발 일정

활동	3	4	5	6	7	8	9	10	11	12	1	2
프로젝트 요약												
프로젝트 계획												
요구사항 정의 및 분석												
Product Backlog												
Sprint Planning Meeting												
Sprint 1												
Sprint 2												
Sprint 3												
Sprint 4												
Sprint 5												
Sprint 6												
Sprint 7												
Sprint 8												
Sprint 9												
Sprint 10												
Daily Scrum Meeting												
Sprint Retrospective												
Sprint Review												
프로젝트 종료												

3. 팀 구성

3.1 팀 구조

본 프로젝트는 1인 개발 프로젝트로 진행되며, 프로젝트 수행에 필요한 기획, 분석, 설계, 구현, 테스트, 문서화 작업을 모두 단일 인원이 담당한다.

비록 팀원 수는 1명이지만, 프로젝트 관리의 효율성을 위해 역할을 세분화하여 수행하며, 각 단계별 산출물 작성과 개발 진행 상황을 스스로 점검하는 방식으로 프로젝트를 운영한다.

따라서 본 프로젝트는 외형적으로는 단일 인원으로 구성되어 있으나, 실제 수행 측면에서는 팀장, 기획자, 개발자, 문서 작성자의 역할을 모두 포함하는 구조로 진행한다.

3.2 역할 및 책임

이름	역할	책임(공동 / 개별)
최수연	팀장	프로젝트의 전반적인 계획 수립, 일정 관리, 진행 상황 점검 및 최종 검토
최수연	기획 및 분석 담당	프로젝트 주제 정의, 요구사항 정의 및 분석, 기능 우선순위 정리
최수연	프론트엔드 개발 담당	React Native 기반 모바일 앱 화면 구성 및 기능 구현
최수연	백엔드 개발 담당	Spring Boot 서버 개발, API 설계 및 MySQL 연동
최수연	지도 API 연동 담당	Naver Maps API(Dynamic Map, GeoCoding, Reverse GeoCoding, Directions5) 연동
최수연	문서 및 형상관리 담당	프로젝트 관련 산출물 작성, GitHub 업로드, 버전 및 변경 이력 관리
최수연	테스트 담당	기능 테스트, 오류 수정, 최종 결과 검토

4. 품질 관리

4.1 팀 미팅 계획

(1) Sprint Planning Meeting

- 팀 미팅 시간 : 매주 목요일 7시
- 토의 내용 : Sprint 목표와 Sprint Backlog를 계획
- 미팅 참석자 : 팀원, 팀장

(2) Daily Scrum Meeting

- 팀 미팅 시간 : 매일 오후 6시에 30분간 진행
- 토의 내용 : 어제 한 일, 오늘 할 일, 장애 현상 등을 공유
- 미팅 참석자 : 팀원, 팀장

(3) Sprint Review

- 팀 미팅 시간 : Sprint 종료 시점
- 토의 내용 : Sprint 기간 동안 팀원 전체가 각자 한 일 리뷰
- 미팅 참석자 : 팀원, 팀장

(4) Sprint Retrospective

- 팀 미팅 시간 : Sprint 종료 시점
- 토의 내용 : Sprint 기간 동안 팀 내부에서 발생한 일들을 회고하며 개선할 것, 잘한 것, 새로 시작할 것을 도출

(5) 비정기 회의

- 프로젝트 진행에 문제가 발생하여 회의가 필요한 경우에 진행된다.
- 비정기 회의는 팀장 또는 지도교수 또는 멘토에 의해 요청될 수 있다.
- 미팅 참석자 : 팀원, 팀장, (지도교수, 멘토)

4.2 변경사항 관리 방법

- 기능 변경을 위해 팀원 전체가 권한을 가져야 하며 다수결의 합의가 필요하다.
- 변경사항이 발생하는 경우 전-후 에 대해 반드시 기록물로 남긴다.
- 관련 산출물은 팀원이 전부 소지한다.
- 기능 변경을 위해서는 팀원 전체의 합의가 필요하지만, 의견 충돌로 진행이 멈출 시에는 팀장이 좀 더 강력한 발언권을 갖는다.
- 변경 사항이 결정 되면 지도 교수님께 반드시 보고하고 경우에 따라 또 다시 변경 될 수 있다.

4.3 산출물 관리 방법

(1) 산출물 식별 및 명명 방법

본 프로젝트의 산출물은 요구사항, 설계, 일정, 테스트, 소스코드로 구분하여 관리한다. 각 산출물은 Git 저장소 내 정해진 디렉터리에 저장하며, 파일명과 버전 정보를 통해 식별 가능하도록 관리한다.

- 요구사항 관련 산출물
 - 과제1.프로젝트정의서 : `/docs/requirements`
 - 과제3.요구사항정의서 : `/docs/requirements`
 - 과제4.요구사항분석서 : `/docs/requirements`
- 설계 관련 산출물
 - 과제5.소프트웨어설계서 : `/docs/design`
- 일정 관련 산출물
 - 과제2.프로젝트관리계획서 : `/docs/plan`
- 테스트 관련 산출물
 - 과제6.인스펙션예제 : `/docs/test`
 - 과제7.테스트결과서 : `/docs/test`
- 소스코드
 - 소스코드 : `/src`

또한 산출물의 버전은 `v주.부.수` 형식으로 관리한다.

예를 들어 `v1.0.0` 은 최초 확정 버전, `v1.1.0` 은 기능 추가, `v1.1.1` 은 오타 및 경미한 수정 사항을 의미한다.

- Major : 요구사항 변경
- Minor : 기능 추가
- Patch : 오타 및 경미한 수정

(2) 산출물의 버전 관리 방법 및 변경 기록 방법

본 프로젝트의 모든 산출물은 Git을 이용하여 형상관리하며, 모든 변경 사항은 `commit` 메시지와 변경 이력을 통해 기록한다.

변경이 필요한 경우에는 변경 필요성을 확인한 뒤, 변경요청서(CR 문서)를 작성하고 별도의 브랜치에서 수정 작업을 수행한 후 `main` 브랜치에 병합한다.

브랜치 전략은 다음과 같다.

- `main` : 최종본 유지
- `draft` : 작업 중 문서 관리
- `review` : 검토 단계 문서 관리
- `experiment` : 실험용 설계안 및 임시 작업 관리

또한 `commit` 메시지는 아래 형식으로 통일하여 기록한다.

- 형식 : [구분자-번호] 변경 내용
- 예시 : [CR-01] 요구사항 3번 수정 - 로그인 정책 변경

구분자는 다음과 같이 사용한다.

- [CR] : 변경 요청
- [FEAT] : 새 기능 추가
- [FIX] : 오류 수정
- [DOCS] : 문서 수정

변경 이력 관리를 위해 `git log`, `git tag`, `CHANGELOG.md` 를 함께 사용하며, 주요 개발 단계에서는 다음과 같은 태그를 부여한다.

- `requirements_freeze` : 요구사항 확정
- `design_complete` : 설계 완료
- `beta_release` : 시제품 완료
- `final_release` : 최종 제출

산출물의 버전 관리와 변경 기록은 본 프로젝트 수행자인 최수연이 직접 담당한다.

(3) 산출물의 저장 및 백업 방법

본 프로젝트의 모든 산출물은 **GitHub 원격 저장소**에 등록하여 관리한다. 문서 산출물과 소스코드는 모두 Git 저장소에 포함시키며, 로컬 저장소와 원격 저장소에 이중으로 보관한다.

- 저장소 관리 방법 : GitHub 원격 저장소 사용
- 저장 대상 : 문서 산출물, 테스트 결과, 소스코드 전체
- 백업 방법 : 로컬 저장 + 원격 저장소 이중 보관
- 동기화 주기 : 최소 주 1회 이상 `push` 수행

이를 통해 산출물의 유실을 방지하고, 변경 이력을 체계적으로 추적할 수 있도록 한다.

5. 개발 환경

5.1 하드웨어 개발 환경

구분	spec
개발용 노트북 / PC	Windows 기반 개발 환경, React Native 앱 개발 및 Spring Boot 서버 실행 가능 사양
테스트용 스마트폰	Android 기반 스마트폰, React Native 앱 실행 및 지도 기능 테스트
서버 실행 환경	백엔드 개발 및 API 테스트가 가능한 로컬 또는 클라우드 서버 환경

5.2 소프트웨어 개발 환경

구분	type	spec
Windows	OS	개발용 운영체제
React Native	Frontend Framework	모바일 애플리케이션 개발
Spring Boot	Backend Framework	서버 및 REST API 개발
MySQL	Database	사용자, 주차장, 결제, 이용 기록 데이터 저장
Naver Maps API	External API	Dynamic Map, GeoCoding, Reverse GeoCoding, Directions5 연동
IntelliJ IDEA	IDE	Spring Boot 백엔드 개발
VS Code / Claude Code	IDE / AI Coding Tool	프론트엔드 개발 및 바이브 코딩 지원
Codex	AI Coding Tool	코드 작성, 수정, 보완 지원
Git / GitHub	Version Control	산출물 및 소스코드 버전 관리

5.3 기타 개발 환경

- 본 프로젝트는 Claude Code와 Codex를 활용한 바이브 코딩 방식을 반영하여 개발을 진행한다.
- 프론트엔드는 React Native, 백엔드는 Spring Boot를 중심으로 개발한다.
- 지도 및 위치 기반 기능은 Naver Maps API를 연동하여 구현한다.
- 산출물과 소스코드는 GitHub 원격 저장소를 통해 관리한다.

6. 산출물

6.1 산출물 정의

본 프로젝트의 산출물은 형상관리 계획에 따라 요구사항, 설계, 일정, 테스트, 소스코드로 구분하여 관리한다.

적용 범위는 다음과 같다.

- 프로젝트정의서
- 요구사항정의서
- 요구사항분석서
- 소프트웨어설계서
- 프로젝트관리계획서
- 인스펙션예제
- 테스트결과서
- 소스코드

본 프로젝트의 형상 항목(Configuration Items)은 다음과 같이 정의한다.

SmartPark 형상 항목 표

구분	형상 항목	파일 위치
요구사항	프로젝트정의서	/docs/requirements
요구사항	요구사항정의서	/docs/requirements
요구사항	요구사항분석서	/docs/requirements
설계	소프트웨어설계서	/docs/design
일정	프로젝트관리계획서	/docs/plan
테스트	인스펙션예제	/docs/test
테스트	테스트결과서	/docs/test
소스코드	소스코드	/src

6.2 산출물 작성일 및 담당자

산출물	최종 작성일	최종 검토자	작성 책임자
프로젝트정의서	작성 완료 시 기입	최수연	최수연
프로젝트관리계획서	작성 완료 시 기입	최수연	최수연
요구사항정의서	작성 완료 시 기입	최수연	최수연
요구사항분석서	작성 완료 시 기입	최수연	최수연
소프트웨어설계서	작성 완료 시 기입	최수연	최수연
인스펙션예제	작성 완료 시 기입	최수연	최수연
테스트결과서	작성 완료 시 기입	최수연	최수연
소스코드	개발 진행 중 지속 반영	최수연	최수연

6.3 기타 제약사항

본 프로젝트의 모든 산출물은 Git 저장소에 등록하여 관리한다.

버전은 v주.부.수 형식으로 관리하며, 변경 사항은 commit 메시지와 CHANGELOG.md를 통해 기록한다.

또한 브랜치는 main, draft, review, experiment 전략에 따라 운영하고, 주요 단계에서는 tag를 사용하여 기준선을 관리한다.

산출물은 로컬과 GitHub 원격 저장소에 이중 보관하며, 최소 주 1회 이상 commit 및 push를 수행한다.

7. 기타 사항

7.1 리스크 관리

No	Risk	Critical	Occur	Mitigation Plan	Alternative	Respon
1	일정 지연	High	Medium	기능 우선순위를 정해 핵심 기능부터 구현하고, 주차 조회·지도 연동·백엔드 API를 먼저 완성한다.	부가 기능(NFC, AI 고도화 등)은 후순위로 조정하여 최소 기능 제품 중심으로 마무리한다.	최수연
2	Naver Maps API 연동 오류	High	Medium	Dynamic Map, GeoCoding, Directions5 기능을 단계적으로 나누어 테스트하고, API 문서와 예제 코드를 기준으로 검증한다.	일부 기능 연동이 지연될 경우 지도 표시와 기본 위치 조회 기능을 우선 제공한다.	최수연
3	프론트엔드-백엔드 연동 문제	High	Medium	API 명세를 먼저 정리하고, 요청/응답 형식을 통일하여 테스트한다.	연동 오류가 지속될 경우 mock 데이터로 프론트엔드를 우선 구현한 뒤 백엔드와 순차적으로 연결한다.	최수연
4	AI 혼잡도 예측 정확도 부족	Medium	Medium	초기에는 단순 규칙 기반 또는 기본 데이터 기반 예측으로 시작하고, 이후 점진적으로 개선한다.	AI 기능이 충분히 구현되지 않을 경우 기본 추천 기능으로 대체한다.	최수연
5	1인 프로젝트로 인한 작업 부담 증가	High	High	작업을 기획, 구현, 테스트, 문서화 단위로 나누어 일정에 맞춰 관리한다.	일정이 부족할 경우 우선순위가 낮은 기능은 축소하거나 문서 범위 내 프로토타입 수준으로 조정한다.	최수연
6	Git 형상관리 누락	Medium	Low	주 1회 이상 commit 및 push를 수행하고, 변경 사항을 commit 메시지와 CHANGELOG에 기록한다.	누락이 발생한 경우 즉시 변경 이력을 정리하고 기준선 태그를 다시 점검한다.	최수연
7	테스트 부족으로 인한 오류 잔존	Medium	Medium	기능별 단위 테스트와 주요 시나리오 테스트를 수행한다.	전체 기능 테스트가 어렵다면 핵심 기능 중심으로 우선 검증한다.	최수연

8. 참고문헌 및 부록

-부록 1 : 시스템정의서 ([최수연]시스템정의서_260313_Doc-001.pdf)

https://drive.google.com/file/d/1PGp9msmd13l4JSmHetwzTGD25mSPFSRp/view?usp=drive_link